

コマ 6: while / for / break / continue

今日のゴール

ループとジャンプ制御を実装する。

第 05 回までは、プログラムは常に先頭から末尾へ 1 回だけ実行された。この回では、条件が成り立つ間、同じ処理を繰り返す `while` と `for` を扱う。

```
int main() {  
    int i;  
    int sum;  
    i = 1;  
    sum = 0;  
    while (i <= 10) {  
        sum = sum + i;  
        i = i + 1;  
    }  
    return sum;  
}
```

目標は、条件判定・反復・途中脱出・スキップを正しく生成できるようにすることである。

1 この回で扱う範囲

対象にするプログラムは、`main` 関数内に以下の要素を含むものに限定する。

種類	例
while 文	<code>while (cond) { ... }</code>
for 文	<code>for (init; cond; step) { ... }</code>
break 文	<code>break;</code>
continue 文	<code>continue;</code>
入れ子ループ	<code>while</code> の中に <code>while</code> など
ループ内の if	<code>while (cond) { if (...) break; }</code>

これまでに扱った変数宣言・代入・算術・比較・if/else は、ループの内部でも使用できる。

関数呼び出し、ポインタ、配列はまだ扱わない。

2 ASTを確認する

2.1 while のAST

```
python3 scaffold/parse_viewer.py sessions/06_loops/tests/target.c
```

このプログラムの内容は次の通り。

```
int main() {  
    int i;  
    int sum;  
    i = 1;  
    sum = 0;  
    while (i <= 10) {  
        sum = sum + i;  
        i = i + 1;  
    }  
    return sum;  
}
```

```
(program  
  (funcdef "main" :type int (params)  
    (block (decl "i" :type int) (decl "sum" :type int)  
      (exprstmt  
        (assign (var "i") (num 1)))  
      (exprstmt  
        (assign (var "sum") (num 0)))  
      (while  
        (cond  
          (le (var "i") (num 10)))  
        (body  
          (block  
            (exprstmt  
              (assign (var "sum")  
                (add (var "sum") (var "i")))))
```

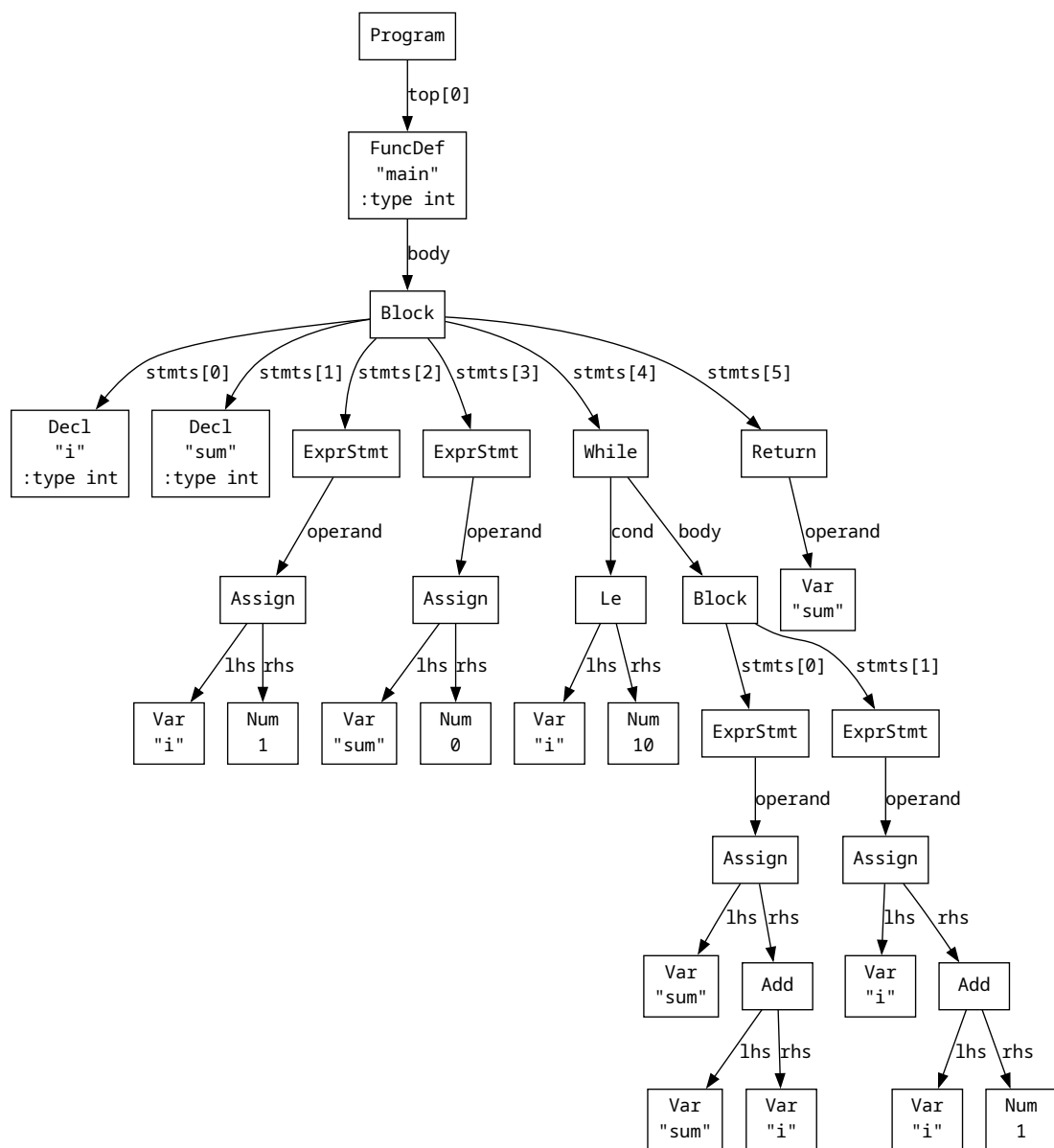


図 1 while ループの AST

```
(exprstmt
  (assign (var "i")
    (add (var "i") (num 1))))))
(return (var "sum"))))
```

while ノードは次のフィールドを持つ。

 フィールド 役割

cond 条件式ノード。0 でなければループを続ける

body ループ本体の文ノード

2.2 for の AST

```
python3 scaffold/parse_viewer.py sessions/06_loops/tests/for_count.c
```

このプログラムの内容は次の通り。

```
int main() {
    int i;
    int sum;
    sum = 0;
    for (i = 0; i < 5; i = i + 1) {
        sum = sum + i;
    }
    return sum;
}
```

```
(program
  (funcdef "main" :type int (params)
    (block (decl "i" :type int) (decl "sum" :type int)
      (exprstmt
        (assign (var "sum") (num 0)))
      (for
        (init
          (assign (var "i") (num 0)))
        (cond
          (lt (var "i") (num 5)))
        (step
          (assign (var "i")
            (add (var "i") (num 1))))
        (body
          (block
            (exprstmt
              (assign (var "sum")
```

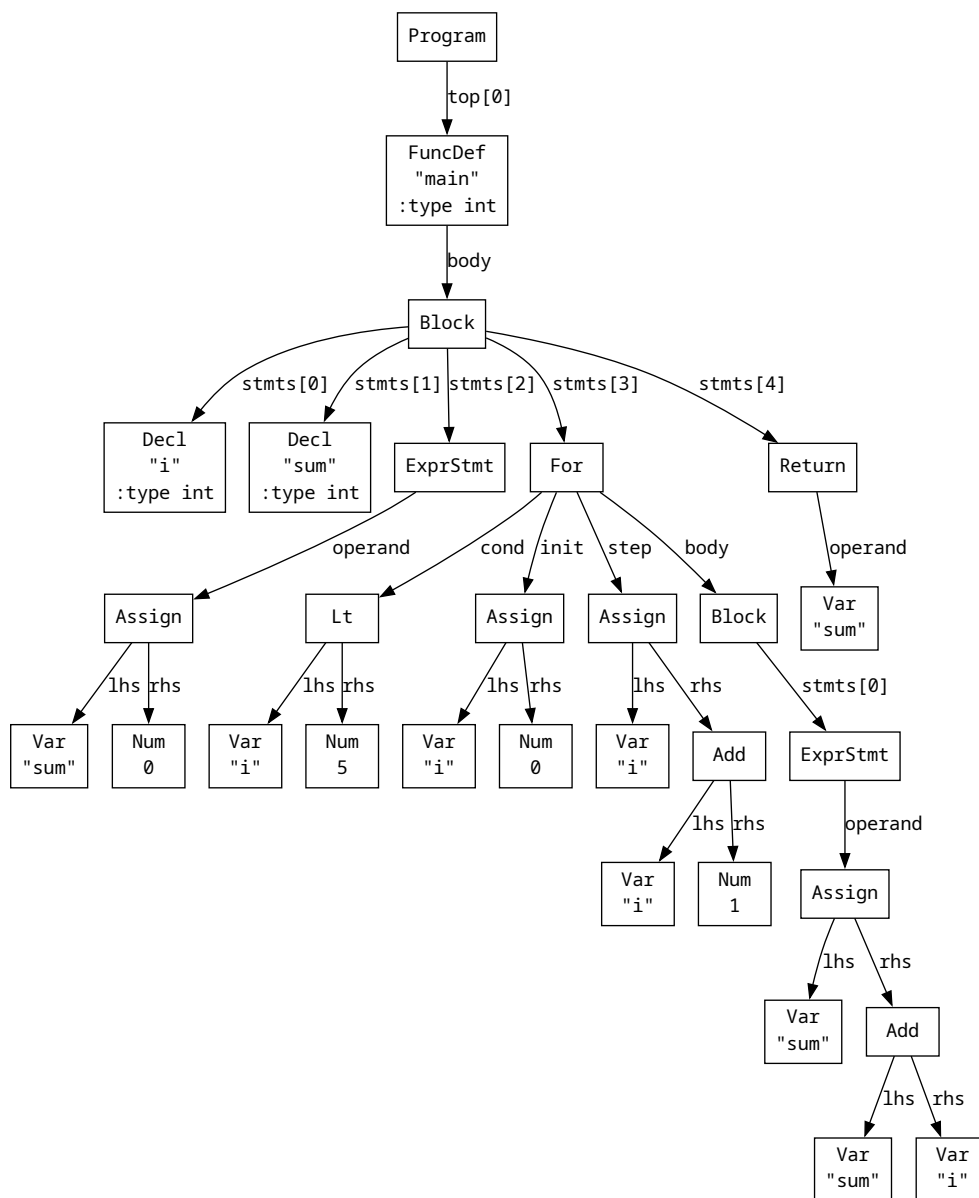


図 2 for ループの AST

```
(add (var "sum") (var "i")))))))
(return (var "sum"))))
```

for ノードは次のフィールドを持つ。

フィールド	役割
init	初期化式ノード。最初に 1 回だけ実行する
cond	条件式ノード。0 でなければループを続ける

フィールド	役割
-------	----

step	ステップ式ノード。各反復の body 実行後に実行する
-------------	------------------------------------

body	ループ本体の文ノード
-------------	------------

`init`、`cond`、`step` は省略可能である。省略された部分は AST 上で `None` になる。たとえば `for (;;)` はすべての要素が `None` になる。

3 while の生成パターン

```
while (cond) {
    body
}
```

これは次の流れに変換する。

```
Lcond:
cond を codegen する
もし cond の結果が 0 なら Lend に飛ぶ
body を gen_stmt する
Lcond に戻る
Lend:
```

アセンブリの形は次のようになる。

```
.Lcond:
# cond -> a0
beqz a0, .Lend
# body
j .Lcond
.Lend:
```

`cond` の計算結果が `a0` に入った状態で `beqz` を置き、偽なら終了ラベルに飛ばす。真なら `body` を実行し、`j` で条件判定に戻る。

4 for の生成パターン

```
for (init; cond; step) {  
    body  
}
```

これは次の流れに変換する。

```
init を codegen する  
Lcond:  
cond を codegen する  
もし cond の結果が 0 なら Lend に飛ぶ  
body を gen_stmt する  
Lstep:  
step を codegen する  
Lcond に戻る  
Lend:
```

アセンブリの形は次のようになる。

```
# init  
.Lcond:  
# cond -> a0  
beqz a0, .Lend  
# body  
.Lstep:  
# step  
j .Lcond  
.Lend:
```

`cond` が省略されている場合は、常に真として扱う。その場合、`beqz` は出さない。

5 break と continue

`break` は、現在のループを終了する文である。コード生成では、現在のループの終了ラベルへジャンプする。

```
j Lend
```

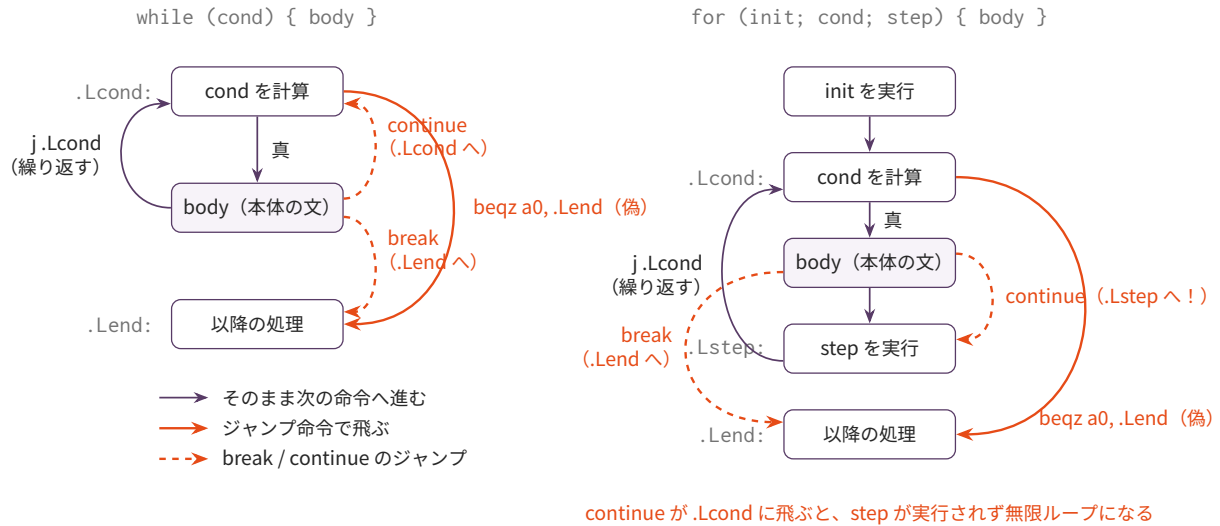


図 3 while / for の制御フローと break / continue の飛び先

`continue` は、現在のループの次の反復へ進む文である。飛び先は `while` と `for` で異なる。

ループ	<code>continue</code> の飛び先
<code>while</code>	条件判定ラベル <code>Lcond</code>
<code>for</code>	<code>step</code> 実行ラベル <code>Lstep</code>

`for` の `continue` が `Lcond` に直接飛んでしまうと、`i = i + 1` のような `step` 処理が実行されず、条件が変わらないまま再チェックされる。その結果、無限ループになることがある。

```
for (i = 1; i <= 10; i = i + 1) {
    if (i % 2 == 0) {
        continue;
    }
    sum = sum + i;
}
```

この例では、`continue` は `Lstep` に飛ぶ必要がある。 `Lcond` に飛ぶと `i` が更新されず、無限ループに陥る。

`break` はどちらのループでも終了ラベルへ飛ぶ。 `continue` の飛び先だけが、`while (Lcond)` と `for (Lstep)` で異なる。

6 ラベルスタック

ループは入れ子になることがある。

```
while (i < 3) {
    while (j < 2) {
        break;
    }
}
```

このとき、内側の `break` は内側のループだけを抜ける必要がある。外側のループまで抜けてはいけない。

ため、現在処理中のループのラベルをスタックで管理する。

インスタンス変数	役割
<code>self._break_stack</code>	現在のループの終了ラベルを積む
<code>self._continue_stack</code>	現在のループの <code>continue</code> 先ラベルを積む

ループのコード生成を始めるときにラベルを `push` し、ループ本体の生成が終わったら `pop` する。

```
# while の例
self._break_stack.append(label_end)
self._continue_stack.append(label_cond)

# body を生成 ...

self._continue_stack.pop()
self._break_stack.pop()
```

`break` や `continue` を生成するときは、スタックの末尾を見る。

```
# break
self.emit(f" j {self._break_stack[-1]}")

# continue
self.emit(f" j {self._continue_stack[-1]}")
```

`break` や `continue` を生成するときは、スタックの末尾を見る。末尾が「現在一番内側のループ」のラベルである。

```
# break
emit(f" j {_break_stack[-1]}")

# continue
emit(f" j {_cont_stack[-1]}")
```

`while` では `continue` 先が `Lcond`、`for` では `continue` 先が `Lstep` になる。ループ本体の生成中に、内側の `break` / `continue` が正しく動作するようになる。

7 `gen_stmt` に追加する処理

node.kind	呼ばれる handler	処理
'While'	<code>gen_stmt_While</code>	条件ラベルと終了ラベルを作る。ラベルスタックに <code>push</code> し、条件判定、 <code>body</code> 、戻りジャンプを生成し、 <code>pop</code> する
'For'	<code>gen_stmt_For</code>	<code>init</code> 、条件、 <code>body</code> 、 <code>step</code> を生成する。ラベルスタックに <code>push/pop</code> も行う

node.kind	呼ばれる handler	処理
'Break'	gen_stmt_Break	self._break_stack[-1] へジャンプ する
'Continue'	gen_stmt_Continue	self._continue_stack[-1] へジャンプ する

8 編集するファイル

- mycc.py

importlib でコマ 5 の Codegen05 を継承した Codegen06 に、以下の要素を追加する。

実装対象	追加する役割
gen_stmt_While(node)	while ループのコード生成
gen_stmt_For(node)	for ループのコード生成
gen_stmt_Break(node)	break 文のコード生成
gen_stmt_Continue(node)	continue 文のコード生成
self._break_stack / self._continue_stack	インスタンス変数を __init__ で追加する

9 tests/

ファイル	内容	期待値
target.c	while で 1..10 の総和	55

ファイル	内容	期待値
<code>for_count.c</code>	for で 0..4 の総和	10
<code>nested_loop.c</code>	while の入れ子 (3×2)	6
<code>break_early.c</code>	break で抜ける	15

`while_sum.c` | while による集計 | 対応する `.ans` を参照 |

`for_fib.c` | for によるフィボナッチ計算 | 対応する `.ans` を参照 |

`break_loop.c` | break を含むループ | 対応する `.ans` を参照 |

`continue_odd.c` | continue を含むループ | 対応する `.ans` を参照 |

10 テスト

```
python3 scaffold/test_runner.py sessions/06_loops
```

`tests/target.c` がコンパイルでき、終了コード 55 になれば基本形は成功。

個別に動かす場合は、次のようにする。

```
python3 sessions/06_loops/mycc.py sessions/06_loops/tests/target.c \
| riscv64-linux-gnu-gcc -x assembler -static - -o out

qemu-riscv64 ./out
echo $?
```

11 注意

この回のテストでは、`break` や `continue` はループの内側にだけ現れる。ループの外側に書かれた `break` や `continue` は扱わない。

`for` の `init`, `cond`, `step` は省略でき、省略時は AST 上で `None` になる。`cond` 省略時は、常に真として扱い、`beqz` を出さないようにする。